

**INTERNATIONAL CYBERSECURITY  
AND DIGITAL FORENSICS ACADEMY**

**INTRO TO CRITICAL INFRASTRUCTURE  
FOR IT LEARNERS**

**LAB 03 — BUILDING YOUR FIRST PLC PROGRAM**

**Student Edition**

Authorized ICDFA training environments only  
[icdfa.edu.ng](http://icdfa.edu.ng)

## Lab 03: Building Your First PLC Program

**Course:** Intro to Critical Infrastructure for IT Learners | **Estimated Duration:** 4–5 hours

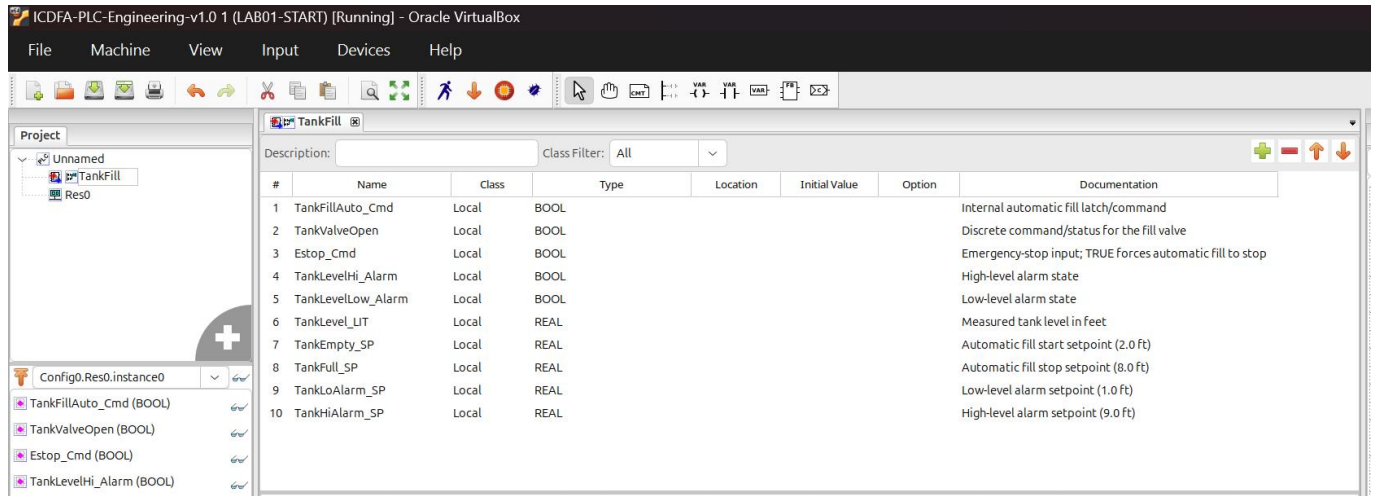
**Submission deadline:** As published in ICDFA E-Campus.

### Procedure

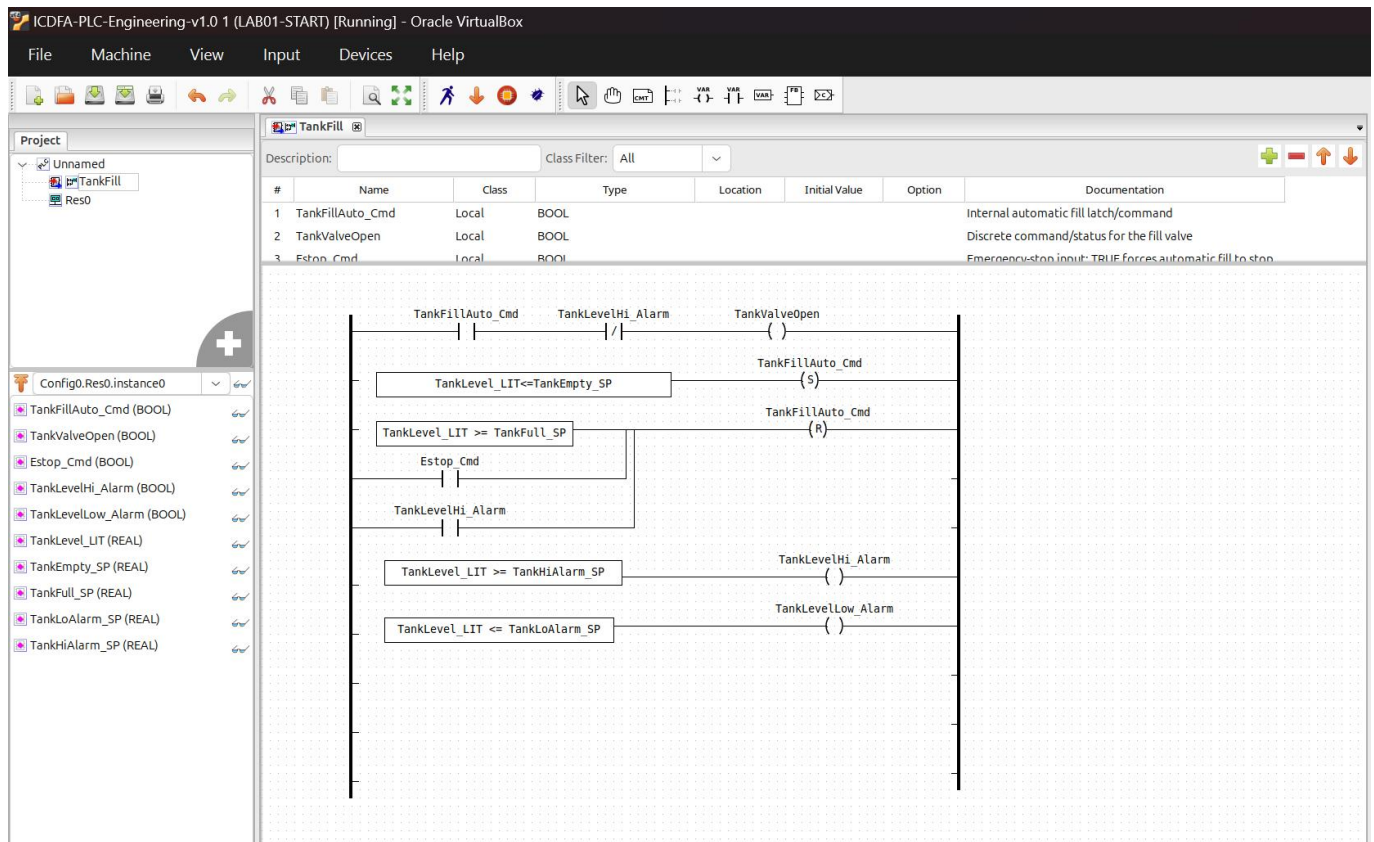
- Step 1.** Start the PLC VM from a clean snapshot. Launch OpenPLC Editor and create a new project folder named TankFill.
- Step 2.** Create a new POU named TankFill and select LD (Ladder Diagram) as the language.
- Step 3.** Add the variables listed in the Variable Plan table in this manual. Use BOOL for discrete states/commands and REAL for tank-level values and setpoints.
- Step 4.** Set the operating values: TankEmpty\_SP = 2.0 ft, TankFull\_SP = 8.0 ft, TankLoAlarm\_SP = 1.0 ft and TankHiAlarm\_SP = 9.0 ft.
- Step 5.** Insert left and right power rails with enough pins for the complete program, then create the first logic rung.
- Step 6.** Build the valve-control rung: TankFillAuto\_Cmd AND NOT TankLevelHi\_Alarm energizes TankValveOpen.
- Step 7.** Build the automatic-start rung: when TankLevel\_LIT <= TankEmpty\_SP, SET TankFillAuto\_Cmd.
- Step 8.** Build the automatic-stop/safety rung: when TankLevel\_LIT >= TankFull\_SP OR Estop\_Cmd is TRUE OR TankLevelHi\_Alarm is TRUE, RESET TankFillAuto\_Cmd.
- Step 9.** Build the high-level alarm rung: TankLevel\_LIT >= TankHiAlarm\_SP energizes TankLevelHi\_Alarm.
- Step 10.** Build the low-level alarm rung: TankLevel\_LIT <= TankLoAlarm\_SP energizes TankLevelLo\_Alarm.
- Step 11.** Check every variable name and data type. Confirm that all contacts/coils/expressions are wired to complete rungs and that no instruction is floating.
- Step 12.** Save the project. If the editor provides validation/compile checking in your VM image, run it and correct any errors before submission.
- Step 13.** Create a logic test table covering at least six scenarios: empty tank, normal filling, full tank, high alarm, low alarm and emergency stop.
- Step 14.** Capture screenshots of the variable table and the completed ladder logic. Submit the project folder together with your written report.

## Required Evidence & Submission

- Screenshot of the completed variable/tag table.



- **Screenshot(s) of the complete ladder program.**



- A six-scenario logic test table showing inputs/conditions and expected outputs.

| Scenario | Condition Description | Input Values                                | Internal Latch State         | Expected Output States                                                                  |
|----------|-----------------------|---------------------------------------------|------------------------------|-----------------------------------------------------------------------------------------|
| 1        | Empty Tank            | TankLevel_LIT = 0.5 ft<br>Estop_Cmd = FALSE | TRUE (Set by Rung 2)         | TankValveOpen = TRUE<br><br>TankLevelLo_Alarm = TRUE<br><br>TankLevelHi_Alarm = FALSE   |
| 2        | Normal Filling        | TankLevel_LIT = 5.0 ft<br>Estop_Cmd = FALSE | TRUE (Retained by Set Latch) | TankValveOpen = TRUE<br><br>TankLevelLo_Alarm = FALSE<br><br>TankLevelHi_Alarm = FALSE  |
| 3        | Full Tank             | TankLevel_LIT = 8.2 ft<br>Estop_Cmd = FALSE | FALSE (Reset by Rung 3)      | TankValveOpen = FALSE<br><br>TankLevelLo_Alarm = FALSE<br><br>TankLevelHi_Alarm = FALSE |
| 4        | High Alarm            | TankLevel_LIT = 9.3 ft<br>Estop_Cmd = FALSE | FALSE (Reset by Rung 3)      | TankValveOpen = FALSE<br><br>TankLevelLo_Alarm = FALSE                                  |

|   |                |                                             |                                |                                                                                         |
|---|----------------|---------------------------------------------|--------------------------------|-----------------------------------------------------------------------------------------|
|   |                |                                             |                                | TankLevelHi_Alarm = TRUE                                                                |
| 5 | Low Alarm      | TankLevel_LIT = 0.8 ft<br>Estop_Cmd = FALSE | TRUE (Set by Rung 2)           | TankValveOpen = TRUE<br><br>TankLevelLo_Alarm = TRUE<br><br>TankLevelHi_Alarm = FALSE   |
| 6 | Emergency Stop | TankLevel_LIT = 4.0 ft<br>Estop_Cmd = TRUE  | FALSE (Reset forced by Rung 3) | TankValveOpen = FALSE<br><br>TankLevelLo_Alarm = FALSE<br><br>TankLevelHi_Alarm = FALSE |

- **A short safety explanation describing why an emergency-stop condition must override normal automatic control.**

**Answer:**

In industrial automation, safety functions like Emergency Stop (Estop\_Cmd) must always take precedence over operational logic. Overriding automatic commands ensures immediate equipment shutdown to prevent catastrophic outcomes, such as fluid overflow, pump cavitating dry, environmental spill, structural overpressurization, or physical injury to personnel. By placing Estop\_Cmd in the Reset Rung (Rung 3) and putting interlocks in series with the valve output coil (Rung 1), the system guarantees a fail-safe state where the valve closes immediately regardless of current tank level conditions.

**Knowledge Check**

1. Why are BOOL and REAL appropriate data types for this project?

Response: The Boolean data type (BOOL) is a binary (discrete) data type, meaning it can only have one of two values; displayed as a 0 or 1 in bits. For example, this type can be used for on/off or true/false logic. While the Real data type (REAL) is a data type that allows for real numbers. These numbers can be either positive or negative whole numbers with a decimal (also known as floating point integers). This type can be used for analog inputs or outputs that require an approximate number within a range of values.

**2. What is the purpose of a SET/RESET pair for TankFillAuto\_Cmd?**

Response: **Response:** A SET/RESET pair creates a memory latch. The SET instruction latches TankFillAuto\_Cmd to TRUE when the tank level drops to the empty setpoint (2.0 ft), keeping the fill process active as the level rises. The RESET instruction turns TankFillAuto\_Cmd back to FALSE once the level hits the full setpoint (8.0 ft), creating a deadband hysteresis cycle between 2.0 ft and 8.0 ft and preventing rapid valve cycling.

**3. What should happen if the tank reaches 9 ft while the fill valve remains commanded open?**

Response: When TankLevel\_LIT reaches 9.0 ft, the high alarm condition (TankLevelHi\_Alarm) energizes. This action immediately triggers two safety measures: it forces a RESET of TankFillAuto\_Cmd via Rung 3 and breaks the series circuit in Rung 1 via the NOT TankLevelHi\_Alarm NC contact. This forces TankValveOpen to FALSE, immediately closing the valve to prevent tank overflow.

**4. Why should PLC logic be validated in simulation before use with physical equipment?**

Response: Validating PLC code in a simulated environment allows developers to catch logical errors, race conditions, incorrect setpoints, or safety flaws without risking damage to expensive physical equipment, causing hazardous material spills, or endangering plant personnel.